

Informe Laboratorio 1

Sección 3

Edicson Solar Salinas
e-mail: edicson.solar@mail.udp.cl

Agosto de 2026

Índice

1. Descripción	2
2. Actividades	2
2.1. Algoritmo de cifrado	2
2.2. Modo stealth	2
2.3. MitM	3
3. Desarrollo de Actividades	4
3.1. Actividad 1	4
3.2. Actividad 2	5
3.3. Actividad 3	13
Conclusiones y comentarios	18

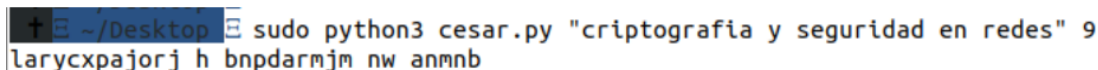
1. Descripción

En este laboratorio se debía esconder un mensaje cifrado dentro de tráfico ICMP parecido al que genera ping. El trabajo se dividió en tres programas. El primero cifra el texto mediante César, el segundo envía un carácter en cada echo request y el tercero recupera el mensaje desde una captura. Los programas fueron pedidos a ChatGPT y luego comprobados mediante ejecuciones y capturas propias.

2. Actividades

2.1. Algoritmo de cifrado

Generar un programa en Python 3 utilizando ChatGPT que permita cifrar texto con el algoritmo César. El texto y el corrimiento deben ingresarse como argumentos.

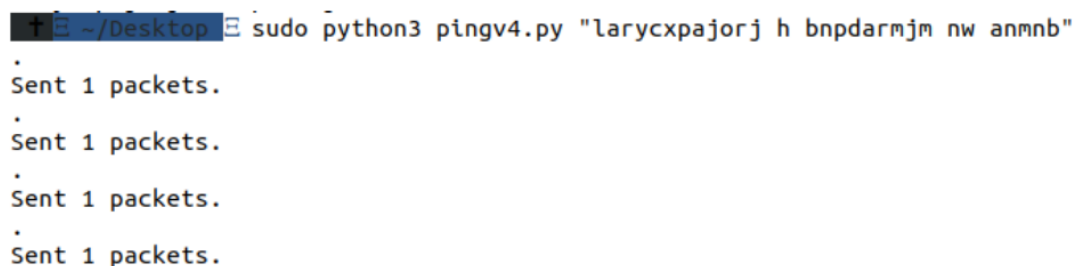


```
+ [~/Desktop] $ sudo python3 cesar.py "criptografia y seguridad en redes" 9
larycxpajorj h bnpdarmjm nw anmnb
```

Figura 1: Enunciado de la actividad 1.

2.2. Modo stealth

Generar un programa en Python 3 utilizando ChatGPT que envíe el texto cifrado mediante paquetes ICMP echo request, usando un carácter por paquete. También se deben comparar los campos con un ping real anterior y posterior.



```
+ [~/Desktop] $ sudo python3 pingv4.py "larycxpajorj h bnpdarmjm nw anmnb"
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
```

Figura 2: Primera parte del enunciado de la actividad 2.

El último carácter del mensaje debe transmitirse como una b.

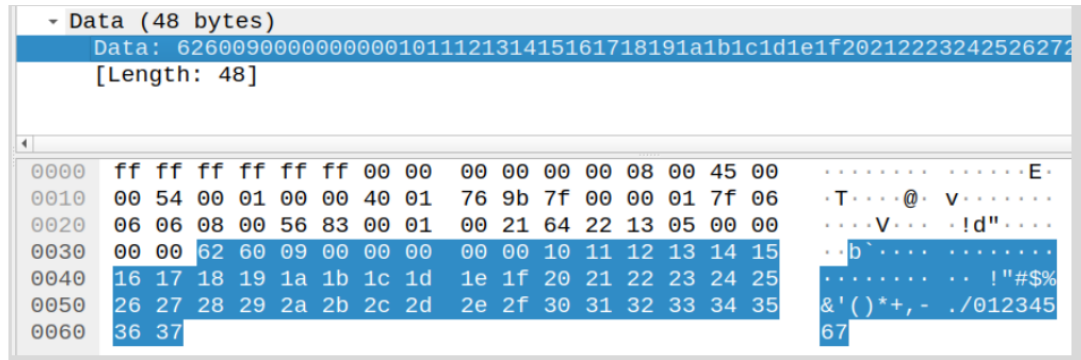


Figura 3: Estructura de referencia incluida en la guía.

2.3. MitM

Generar un programa en Python 3 utilizando ChatGPT que lea el mensaje transmitido, pruebe todos los corrimientos y marque en verde la alternativa más probable.

```

$ sudo python3 readv2.py cesar.pcapng
0   larycxpajorj h bnpdarmjm nw anmnb
1   kzqxbwozinqi g amoczqlil mv zmlma
2   jypwavnyhmpf f zlnbypkhk lu yklkz
3   ixovzumxglog e ykmaxojgj kt xkjky
4   hwnuytlwfkf d xjlzwnifi js wjijx
5   gvmtxskvejme c wikyvmeheh ir vihiw
6   fulswrjudild b vhjxulgdg hq uhghv
7   etkrvqitchkc a ugiwtkfcf gp tgfgu
8   dsjquphsbgjb z tfhvsjebe fo sfef
9   criptografia y seguridad en redes
10  bqhosnfqzehz x rdftqhczc dm qdcdr
11  apgnrmepydgy w qcespgbyb cl pcabcq
12  zofmqldoxcfx v pbdrofaxa bk obabp
13  ynelpkcnwbew u oacqnezwz aj nazao
14  xmdkojbmadv t nzbpmdivy zi mzyzn
15  wlcjnia luzcu s myaolcxux yh lyxym
16  vkbimhzktybt r lxznkbtw xg kxwxl
17  ujahlgysxas q kwymjavsv wf jwvkw
18  tizgkfxirwzr p jvxlizuru ve ivuvj
19  shyfjewhqvyq o iuwkhytqt ud hutui
20  rgxeidvgpuxp n htvjgxspc tc gtsth
21  qfwdhcufotwo m gsuifwr or sb fsrsg
22  pevcbtensvn l frthevqnq ra erqrf
23  odubfasdmrum k eqsgdupmp qz dqpqe
24  nctaezrclqtl j dprfctolo py cpopd
25  mbszdyqbksk i coqebnkn ox bonoc

```

Figura 4: Enunciado de la actividad 3.

Finalmente, se deben explicar cuatro problemas encontrados al trabajar con ChatGPT y cómo fueron resueltos.

3. Desarrollo de Actividades

3.1. Actividad 1

El programa debía recibir todo desde la terminal. Por eso en el prompt indiqué la forma exacta de ejecución y pedí que la frase y la llave no quedaran escritas dentro del archivo. La figura 5 corresponde al mensaje real enviado a ChatGPT.

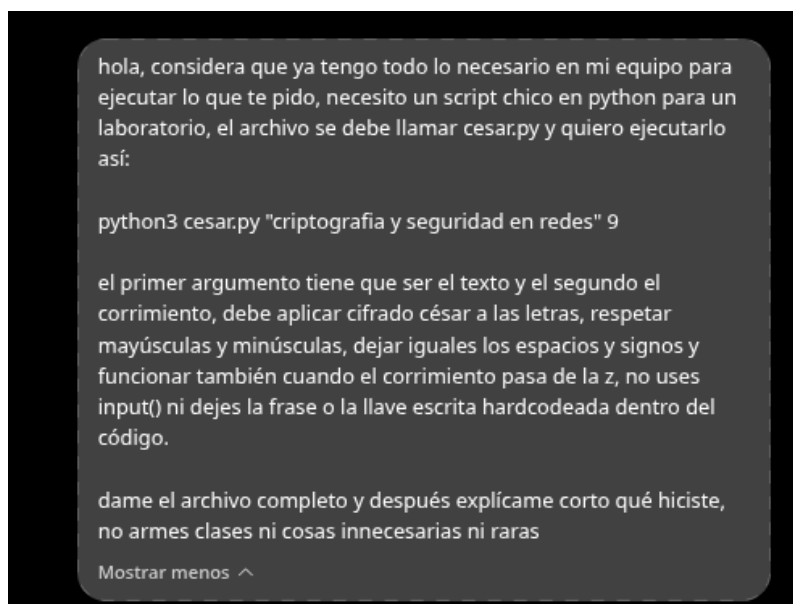


Figura 5: Prompt usado para generar `cesar.py`.

El código revisa que existan dos argumentos, convierte el corrimiento a entero y desplaza solamente las letras. Para volver desde la `z` hacia el comienzo del alfabeto usa módulo 26. Los espacios y signos pasan directamente al resultado.

En la primera ejecución usé el texto de la guía y corrimiento 9. El resultado fue `larycxpajorj h bnpdarmjm nw anmnb`.

```
(venv) lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab1$ python3 cesar.py "criptografia y seguridad en redes" 9
larycxpajorj h bnpdarmjm nw anmnb
(venv) lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab1$
```

Figura 6: Ejecución de César con el texto y el corrimiento recibidos por terminal.

```
(venv) lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab1$ python3 cesar.py "xyz XYZ" 3
abc ABC
(venv) lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab1$
```

Figura 7: Prueba adicional del cambio desde z hacia a.

También probé xyz XYZ con corrimiento 3. La salida fue abc ABC, con lo que pude revisar la vuelta del alfabeto y las mayúsculas sin depender del ejemplo principal.

El criptograma final contiene 33 caracteres contando los espacios. Ese número determina la cantidad de solicitudes enviadas en la actividad siguiente.

3.2. Actividad 2

Ping de referencia

Hice la captura correcta en la interfaz lo. El comando de la figura 8 confirma que el destino fue 127.0.0.1 y que el ping normal trabajó con 56 bytes de datos, TTL 64 y sin pérdida de paquetes.

```
(venv) lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab1$ ping -c 3 127.0.0.1
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.082 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.046 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.041 ms

--- 127.0.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2036ms
rtt min/avg/max/mdev = 0.041/0.056/0.082/0.018 ms
```

Figura 8: Ping usado como referencia antes de construir los paquetes.



```

▶ Frame 3: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface lo, id 0
▶ Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
▼ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 84
    Identification: 0xbcd8 (48344)
  ▶ 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: ICMP (1)
    Header Checksum: 0x7fce [correct]
    [Header checksum status: Good]
    [Calculated Checksum: 0x7fce]
    Source Address: 127.0.0.1
    Destination Address: 127.0.0.1
  ▼ Internet Control Message Protocol
    Type: 8 (Echo (ping) request)
    Code: 0
    Checksum: 0xdc26 [correct]
    [Checksum Status: Good]
    Identifier (BE): 31697 (0x7bd1)
    Identifier (LE): 53627 (0xd17b)
    Sequence Number (BE): 2 (0x0002)
    Sequence Number (LE): 512 (0x0200)
    [Response frame: 4]
    Timestamp from icmp data: Aug 29, 2026 23:18:11.000000000 -04
    [Timestamp from icmp data (relative): 0.862057057 seconds]
  ▼ Data (48 bytes)
    Data: 4d270d0000000000101112131415161718191a1b1c1d1e1f202122232425262728292a2b...
    [Length: 48]

0000 00 00 00 00 00 00 00 00 00 00 00 08 00 45 00 .....E.
0010 00 54 bc d8 40 00 40 01 7f ce 7f 00 00 01 7f 00 .T..@. ....
0020 00 01 08 00 dc 26 7b d1 00 02 f3 a0 93 6a 00 00 ..&{. ....
0030 00 00 4d 27 0d 00 00 00 00 00 10 11 12 13 14 15 [M].....
0040 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25 .....!#$%
0050 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35 &'()*+,-./012345
0060 36 37 67

```

Figura 9: Echo request real anterior al mensaje.

Para la prueba final envié dos solicitudes normales, ejecuté el programa stealth y luego envié otras dos solicitudes normales. En la figura 9 está abierto uno de los echo request anteriores. Se ve la interfaz lo, el tipo 8, DF activo, largo IPv4 de 84 bytes, TTL 64 y ambos checksums correctos.

Generación de stealth.py

Con los campos del ping ya ubicados, entregué a ChatGPT el prompt de la figura 10. El mensaje especifica el lugar del carácter y también la estructura que debía conservar el resto de Data.

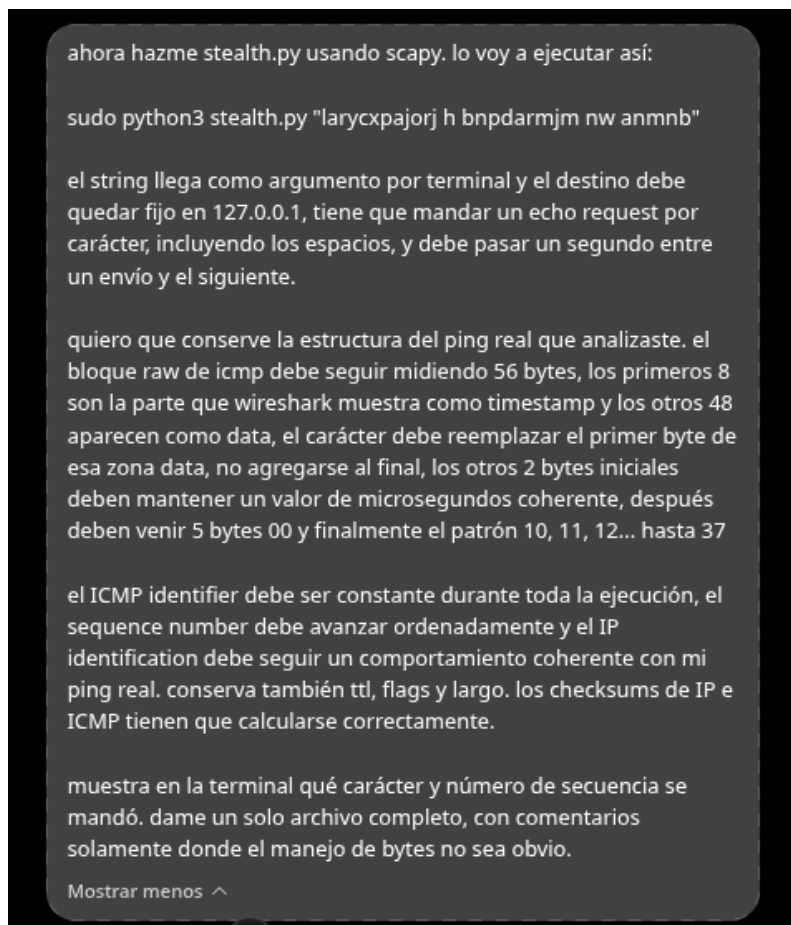


Figura 10: Prompt completo usado para generar `stealth.py`.

La versión final recibe el criptograma como único argumento. En cada iteración arma un paquete con estas partes:

- IPv4 desde 127.0.0.1 hacia 127.0.0.1, TTL 64, DF y un Identification que aumenta en uno.
- ICMP tipo 8 con un Identifier fijo durante la ejecución y Sequence Number desde 1 hasta 33.
- Ocho bytes de timestamp seguidos por 48 bytes de Data.
- En Data aparece el carácter, dos bytes relacionados con los microsegundos, cinco bytes 00 y el patrón desde 0x10 hasta 0x37.
- Una espera de un segundo antes del siguiente carácter.

No fijé los checksums manualmente. Scapy los calculó después de completar el paquete.

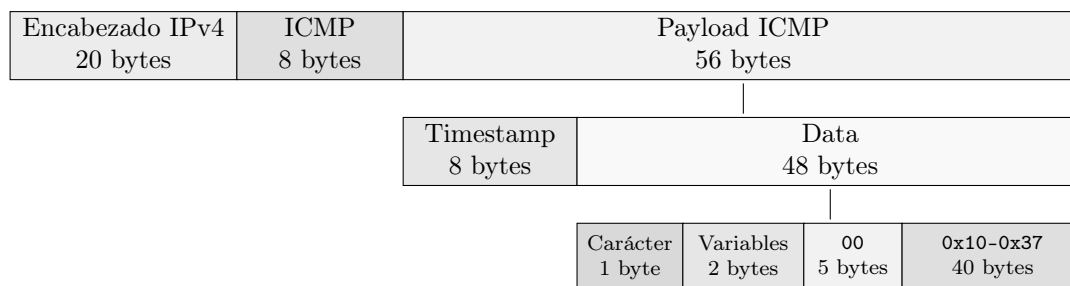


Figura 11: Distribución de los 84 bytes del datagrama IPv4 generado. El dibujo no está a escala.

La terminal registró las 33 iteraciones. La última corresponde a la secuencia 33 y al carácter b.

```
(venv) lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab1$ sudo python3 stealth.py "larycxpajorj h bnpdarmjm nw anmnb"
[sudo] password for lavapatos:
seq=1 caracter='l'
seq=2 caracter='a'
seq=3 caracter='r'
seq=4 caracter='y'
seq=5 caracter='c'
seq=6 caracter='x'
seq=7 caracter='p'
seq=8 caracter='a'
seq=9 caracter='j'
seq=10 caracter='o'
seq=11 caracter='r'
seq=12 caracter='j'
seq=13 caracter=' '
seq=14 caracter='h'
seq=15 caracter=' '
seq=16 caracter='b'
seq=17 caracter='n'
seq=18 caracter='p'
seq=19 caracter='d'
seq=20 caracter='a'
seq=21 caracter='r'
seq=22 caracter='m'
seq=23 caracter='j'
seq=24 caracter='m'
seq=25 caracter=' '
seq=26 caracter='n'
seq=27 caracter='w'
seq=28 caracter=' '
seq=29 caracter='a'
seq=30 caracter='n'
seq=31 caracter='m'
seq=32 caracter='n'
seq=33 caracter='b'
(venv) lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab1$
```

Figura 12: Ejecución completa del programa stealth.

Al filtrar solamente los echo request aparecen dos solicitudes normales, las 33 del mensaje y otras dos solicitudes normales. El bloque central mantiene el Identifier 32921, mientras la secuencia y el IP Identification avanzan de manera ordenada.

No.	Time	Source	Destination	Protocol	Length	Info	Delta time	IP ID	ICMP ID	ICMP seq
1	0.000000	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x7bd1, seq=1/256, ttl...	0.000000000	0xbc62	31697	0...1 (0x0001)
3	1.023290	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x7bd1, seq=2/512, ttl...	1.023257251	0xbcd8	31697	0...2 (0x0002)
5	23.33326	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=1/256, ttl...	23.333246004	0xf607	32921	0...1 (0x0001)
6	1.058503	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=2/512, ttl...	1.058503883	0xf608	32921	0...2 (0x0002)
7	1.033758	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=3/768, ttl...	1.033758884	0xf609	32921	0...3 (0x0003)
8	1.059436	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=4/1024, ttl...	1.059436962	0xf60a	32921	0...4 (0x0004)
9	1.055029	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=5/1280, ttl...	1.055029229	0xf60b	32921	0...5 (0x0005)
10	1.065992	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=6/1536, ttl...	1.065992088	0xf60c	32921	0...6 (0x0006)
11	1.054955	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=7/1792, ttl...	1.054955316	0xf60d	32921	0...7 (0x0007)
12	1.038111	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=8/2048, ttl...	1.038111512	0xf60e	32921	0...8 (0x0008)
13	1.052305	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=9/2304, ttl...	1.052305685	0xf60f	32921	0...9 (0x0009)
14	1.068774	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=10/2560, ttl...	1.068774973	0xf610	32921	0...10 (0x000a)
15	1.068654	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=11/2816, ttl...	1.068654933	0xf611	32921	0...11 (0x000b)
16	1.065361	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=12/3072, ttl...	1.065361844	0xf612	32921	0...12 (0x000c)
17	1.039264	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=13/3328, ttl...	1.039264593	0xf613	32921	0...13 (0x000d)
18	1.048842	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=14/3584, ttl...	1.048842730	0xf614	32921	0...14 (0x000e)
19	1.044578	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=15/3840, ttl...	1.044578343	0xf615	32921	0...15 (0x000f)
20	1.037771	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=16/4096, ttl...	1.037771470	0xf616	32921	0...16 (0x0010)
21	1.032554	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=17/4352, ttl...	1.032554424	0xf617	32921	0...17 (0x0011)
22	1.056172	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=18/4608, ttl...	1.056172771	0xf618	32921	0...18 (0x0012)
23	1.047076	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=19/4864, ttl...	1.047076891	0xf619	32921	0...19 (0x0013)
24	1.027933	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=20/5120, ttl...	1.027933568	0xf61a	32921	0...20 (0x0014)
25	1.057766	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=21/5376, ttl...	1.057766666	0xf61b	32921	0...21 (0x0015)
26	1.052053	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=22/5632, ttl...	1.052053156	0xf61c	32921	0...22 (0x0016)
27	1.040082	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=23/5888, ttl...	1.040082108	0xf61d	32921	0...23 (0x0017)
28	1.037564	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=24/6144, ttl...	1.037564463	0xf61e	32921	0...24 (0x0018)
29	1.052158	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=25/6400, ttl...	1.052158035	0xf61f	32921	0...25 (0x0019)
30	1.053397	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=26/6656, ttl...	1.053397069	0xf620	32921	0...26 (0x001a)
31	1.056633	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=27/6912, ttl...	1.056633526	0xf621	32921	0...27 (0x001b)
32	1.050264	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=28/7168, ttl...	1.050264133	0xf622	32921	0...28 (0x001c)
33	1.062158	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=29/7424, ttl...	1.062158873	0xf623	32921	0...29 (0x001d)
34	1.057447	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=30/7680, ttl...	1.057447532	0xf624	32921	0...30 (0x001e)
35	1.044920	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=31/7936, ttl...	1.044920750	0xf625	32921	0...31 (0x001f)
36	1.056821	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=32/8192, ttl...	1.056821036	0xf626	32921	0...32 (0x0020)
37	1.047196	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8099, seq=33/8448, ttl...	1.047196875	0xf627	32921	0...33 (0x0021)
38	24.87485	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8e44, seq=1/256, ttl...	24.874851336	0xc4bf	36420	0...1 (0x0001)
40	1.016295	127.0.0.1	127.0.0.1	ICMP	98	Echo (ping) request id=0x8e44, seq=2/512, ttl...	1.016256206	0xc5aa	36420	0...2 (0x0002)

Figura 13: Orden de los echo request de la captura final.

Tabla 1: Campos de los tres grupos de echo request.

Campo	Ping anterior	Stealth	Ping posterior
Solicitudes	2	33	2
IP origen y destino	Loopback	Loopback	Loopback
TTL / largo IPv4	64 / 84 bytes	64 / 84 bytes	64 / 84 bytes
Bandera	DF	DF	DF
ICMP Identifier	31697	32921	36420
Sequence Number	1 a 2	1 a 33	1 a 2
IP Identification	Valores del kernel	0xf607 a 0xf627	Valores del kernel
Data	48 bytes	48 bytes	48 bytes

Los Identifier cambian entre grupos porque cada ping y el programa stealth son procesos distintos. Dentro de cada ejecución se mantienen constantes, que es el comportamiento que interesaba comparar.

Detalle de los bytes

```

▶ Frame 16: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface lo, id 0
▶ Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
▼ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
        Total Length: 84
        Identification: 0xf612 (62994)
    ▶ 010. .... = Flags: 0x2, Don't fragment
        ...0 0000 0000 0000 = Fragment Offset: 0
        Time to Live: 64
        Protocol: ICMP (1)
        Header Checksum: 0x4694 [correct]
        [Header checksum status: Good]
        [Calculated Checksum: 0x4694]
        Source Address: 127.0.0.1
        Destination Address: 127.0.0.1
    ▼ Internet Control Message Protocol
        Type: 8 (Echo (ping) request)
        Code: 0
        Checksum: 0x98ab [correct]
        [Checksum Status: Good]
        Identifier (BE): 32921 (0x8099)
        Identifier (LE): 39296 (0x9980)
        Sequence Number (BE): 12 (0x000c)
        Sequence Number (LE): 3072 (0x0c00)
        ▶ [No response seen]
        Timestamp from icmp data: Aug 29, 2026 23:18:46.000000000 -04
        [Timestamp from icmp data (relative): 0.816202686 seconds]
    ▼ Data (48 bytes)
        Data: 6ad00b0000000000101112131415161718191a1b1c1d1e1f202122232425262728292a2b...
        [Length: 48]

0000  ff ff ff ff ff 00 00 00 00 00 08 00 45 00  .....E.
0010  00 54 f6 12 40 00 40 01 46 94 7f 00 00 01 7f 00  .T..@. F.....
0020  00 01 08 00 98 ab 80 99 00 0c 16 a1 93 6a 00 00  .....j..
0030  00 00 6a d0 0b 00 00 00 00 00 10 11 12 13 14 15  ..j.....
0040  16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25  .....! "$%
0050  26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35  &'()*+,-./012345
0060  36 37                                           67

```

Figura 14: Detalle de un paquete generado por `stealth.py`.

El paquete de la figura 14 corresponde a la secuencia 12. Wireshark reconoce el timestamp y muestra 48 bytes de Data. Ese bloque comienza con `6a d0 0b 00 00 00 00 00`. El byte `0x6a` es la letra `j`. Después aparecen los otros dos bytes variables, cinco ceros y el patrón completo `10 11 12 ... 37`. En la misma captura figuran como correctos los checksums IPv4 e ICMP.

El resaltado amarillo corresponde al aviso `[No response seen]`. Los echo request de los ping reales tienen sus echo reply dentro del PCAP. Los 33 paquetes generados no recibieron respuesta. Los campos evaluados coinciden con el ping, pero la falta de respuestas permite distinguir el intercambio completo.

```

▶ Frame 37: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface lo, id 0
▶ Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
▼ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
      Total Length: 84
      Identification: 0xf627 (63015)
    ▶ 010. .... = Flags: 0x2, Don't fragment
      ...0 0000 0000 0000 = Fragment Offset: 0
      Time to Live: 64
      Protocol: ICMP (1)
      Header Checksum: 0x467f [correct]
      [Header checksum status: Good]
      [Calculated Checksum: 0x467f]
      Source Address: 127.0.0.1
      Destination Address: 127.0.0.1
    ▼ Internet Control Message Protocol
      Type: 8 (Echo (ping) request)
      Code: 0
      Checksum: 0x8a48 [correct]
      [Checksum Status: Good]
      Identifier (BE): 32921 (0x8099)
      Identifier (LE): 39296 (0x9980)
      Sequence Number (BE): 33 (0x0021)
      Sequence Number (LE): 8448 (0x2100)
    ▶ [No response seen]
      Timestamp from icmp data: Aug 29, 2026 23:19:08.000000000 -04
      [Timestamp from icmp data (relative): 0.818861698 seconds]
    ▼ Data (48 bytes)
      Data: 621e0c0000000000101112131415161718191a1b1c1d1e1f202122232425262728292a2b...
      [Length: 48]

```

0000	ff ff ff ff ff ff 00 00	00 00 00 00 08 00 45 00	E.
0010	00 54 f6 27 40 00 40 01	46 7f 7f 00 00 01 7f 00	..T'@. F.....
0020	00 01 08 00 8a 48 80 99	00 21 2c a1 93 6a 00 00	H. .!,.j..
0030	00 00 62 1e 0c 00 00 00	00 00 10 11 12 13 14 15	..b.....
0040	16 17 18 19 1a 1b 1c 1d	1e 1f 20 21 22 23 24 25	 !"#%\$
0050	26 27 28 29 2a 2b 2c 2d	2e 2f 30 31 32 33 34 35	&'()*+,-./012345
0060	36 37		67

Figura 15: Último carácter guardado dentro del PCAP.

En el último paquete Data comienza con 0x62, que corresponde a la letra b. La secuencia visible es 33 y el IP Identification continúa el orden del bloque anterior.

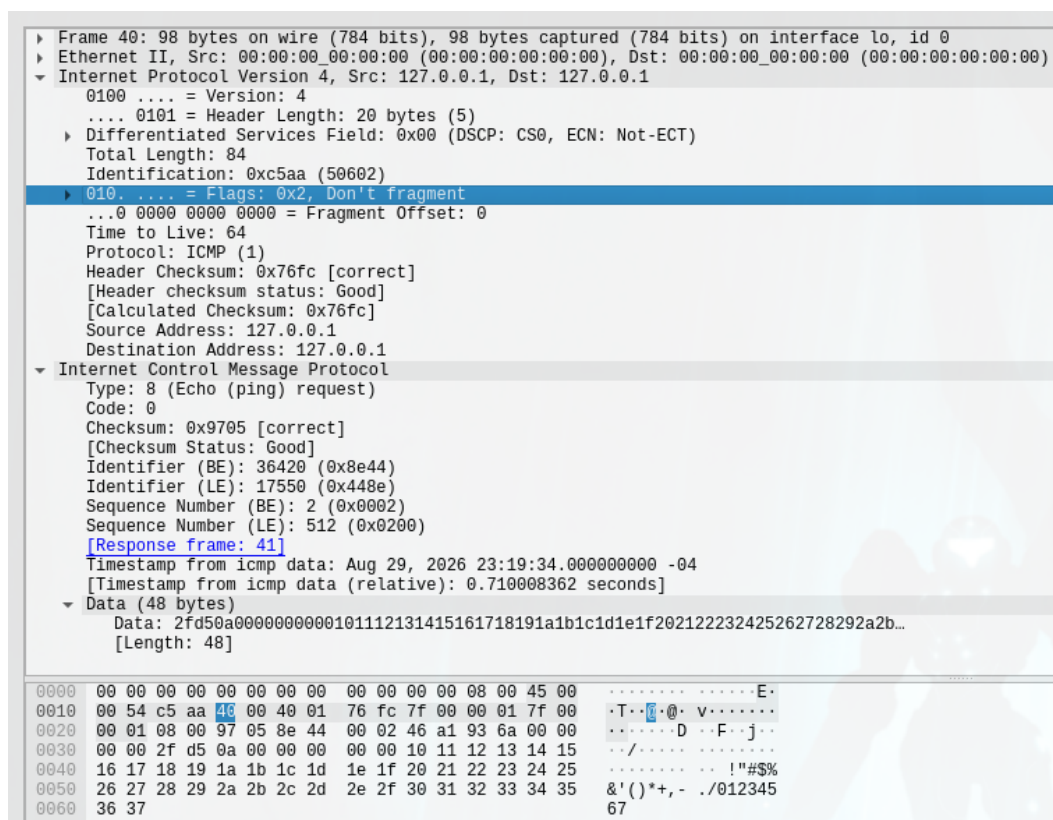


Figura 16: Echo request real posterior al mensaje.

Después del mensaje repetí el ping. La figura 16 vuelve a mostrar loopback, TTL 64, 84 bytes, DF y checksums correctos. Los identificadores son distintos porque este ping se ejecutó después y corresponde a otro proceso.

Intervalos y mensaje transportado

5	0.000000000	0xf607	32921	1
6	1.058503883	0xf608	32921	2
7	1.033758884	0xf609	32921	3
8	1.059436962	0xf60a	32921	4
9	1.055029229	0xf60b	32921	5
10	1.065992080	0xf60c	32921	6
11	1.054955316	0xf60d	32921	7
12	1.038111512	0xf60e	32921	8
13	1.052305685	0xf60f	32921	9
14	1.068774973	0xf610	32921	10
15	1.068654933	0xf611	32921	11
16	1.065361844	0xf612	32921	12
17	1.039264593	0xf613	32921	13
18	1.048842730	0xf614	32921	14
19	1.044578343	0xf615	32921	15
20	1.037771470	0xf616	32921	16
21	1.032554424	0xf617	32921	17
22	1.056172771	0xf618	32921	18
23	1.047076891	0xf619	32921	19
24	1.027933568	0xf61a	32921	20
25	1.057766666	0xf61b	32921	21
26	1.052053156	0xf61c	32921	22
27	1.040082108	0xf61d	32921	23
28	1.037564463	0xf61e	32921	24
29	1.052158035	0xf61f	32921	25
30	1.053397069	0xf620	32921	26
31	1.056633526	0xf621	32921	27
32	1.050264133	0xf622	32921	28
33	1.062158873	0xf623	32921	29
34	1.057447532	0xf624	32921	30
35	1.044920750	0xf625	32921	31
36	1.056821036	0xf626	32921	32
37	1.047196875	0xf627	32921	33

Figura 17: Intervalos cercanos a un segundo entre las 33 solicitudes.

Los 32 intervalos entre paquetes stealth quedaron entre 1,028 y 1,069 segundos, con un promedio de 1,051 segundos. La diferencia incluye el segundo de espera y el tiempo que Scapy demora en preparar y enviar cada paquete.

Al tomar el primer byte de Data en orden se reconstruye `larycxpajorj h bnpdarmjm nw anmnb`. También se revisaron los 33 paquetes activando la validación de Wireshark. Los checksums IPv4 e ICMP fueron correctos en todos.

3.3. Actividad 3

La figura 18 contiene el prompt real de esta parte. Pedí que el programa leyera el PCAP desde una ruta entregada por terminal, ignorara los echo reply y probara todas las llaves sin dejar escrita la respuesta dentro del código.

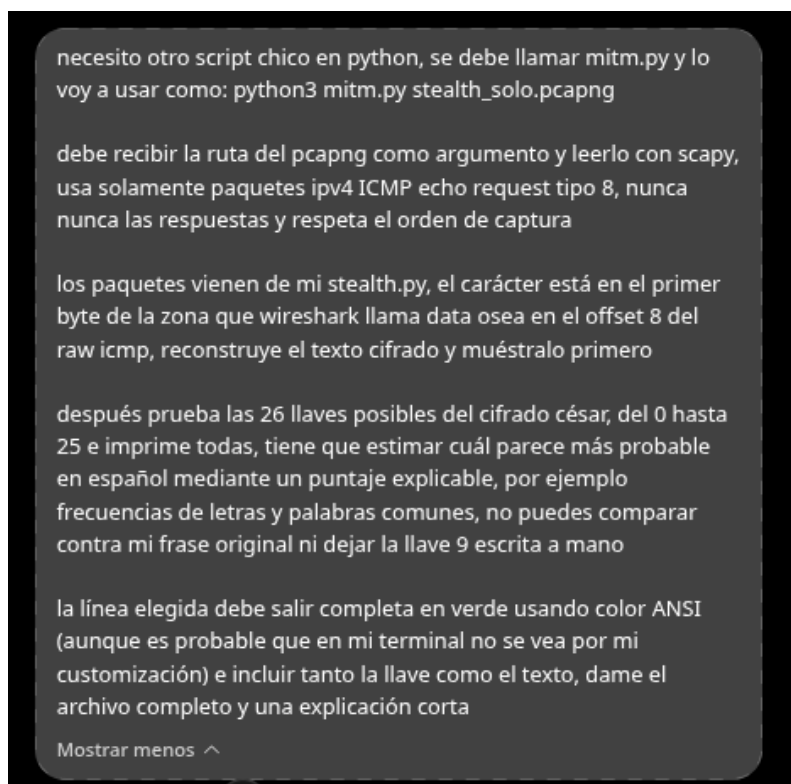


Figura 18: Prompt utilizado para generar mitm.py.

El programa usa `PcapNgReader` y conserva solamente IPv4 ICMP tipo 8. Luego salta los ocho bytes del timestamp y toma el primer byte de Data. Con esos bytes recompone el criptograma en el mismo orden de la captura.

La ruta del PCAP aparece como argumento en la figura 19. La primera salida ya corresponde al mismo criptograma obtenido con César.

```
(venv) lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab1$ python3 mitm.py material_lab1/pcaps/stealth_solo_v3.pcapng
Texto cifrado: larycxpajorj h bnpdarmjm nw anmnb
```

Figura 19: Lectura del PCAP y recuperación del texto cifrado.

Después se prueban las llaves desde 0 hasta 25. Para escoger una alternativa, el código ocupa frecuencias de letras, algunas palabras comunes y una proporción esperada de vocales. No compara contra la frase original ni fija la llave 9. La alternativa con mayor puntaje aparece en verde con el texto `criptografia y seguridad en redes`.

```

Puntaje: frecuencias de letras + palabras comunes + cercanía a una proporción normal de vocales.

llave 0 | puntaje -109.36 | larycxpajorj h bnpdarmjm nw anmnb
llave 1 | puntaje -121.11 | kzqxbwozinqi g amoczqlil mv zmlma
llave 2 | puntaje -140.18 | jypwavyhmpf f zlnbypkhk lu yklkz
llave 3 | puntaje -140.90 | ixovzumxglog e ykmaxojgj kt xkjky
llave 4 | puntaje -147.01 | hwnuytlwfknf d xjlnwnifi js wjijx
llave 5 | puntaje -131.31 | gvmtxskvejme c wikyvmeh ir vihiw
llave 6 | puntaje -124.97 | fulswrjudild b vhjxulgdg hq uhghv
llave 7 | puntaje -129.32 | etkrvqitchkc a ugiwtkfcf gp tgfgu
llave 8 | puntaje -115.93 | dsjquphsbgjb z tfhvsjebe fo sfegt
llave 9 | puntaje -69.30 | criptografia y seguridad en redes
llave 10 | puntaje -116.58 | bqhosnfqzehz x rdftqhczc dm qdcdr
llave 11 | puntaje -113.76 | apgnrmepdygy w qcespgbyb cl pcbcq
llave 12 | puntaje -117.28 | zofmqldoxcfx v pbdrofaxa bk obabp
llave 13 | puntaje -118.33 | ynelpkcnwbew u oacqnezwz aj nazao
llave 14 | puntaje -123.83 | xmdkojbmadv t nzbpmdivy zi mzyzn
llave 15 | puntaje -116.98 | wlcjnia luzcu s myaolcxux yh lyxym
llave 16 | puntaje -160.25 | vkbimhzktybt r lxznkbtw xg kxwxl
llave 17 | puntaje -151.78 | ujahlgysxas q kwymjavsv wf jwvwk
llave 18 | puntaje -127.68 | tizgkfxirwzr p jvxlizuru ve ivuvj
llave 19 | puntaje -129.83 | shyfjewhqvyq o iuwkhytqt ud hutui
llave 20 | puntaje -115.87 | rgxeidvgpuxp n htvjgxspc tc gtsth
llave 21 | puntaje -120.75 | qfwdhucufotwo m gsuifwrwr sb fsrsg
llave 22 | puntaje -102.70 | pevcbtensvn l frthevqng ra erqrf
llave 23 | puntaje -107.76 | odubfasdmrum k eqsgdupmp qz dqpqe
llave 24 | puntaje -98.78 | nctaezrclqtl j dprfcto lo py cpopd
llave 25 | puntaje -117.37 | mbszdyqbksk i coqebnkn ox bonoc

```

Figura 20: Las 26 posibilidades y la llave 9 marcada en verde.

Problemas encontrados

1. **Capturé la interfaz equivocada.** El primer PCAP se tomó en wlp2s0 aunque el comando visible era un ping a loopback. ChatGPT notó que las direcciones pertenecían a la red local y no pudo relacionar esos bytes con 127.0.0.1. Repetí la captura directamente en lo y comprobé primero las direcciones. La figura 21 muestra el mensaje inicial y la corrección que envié.

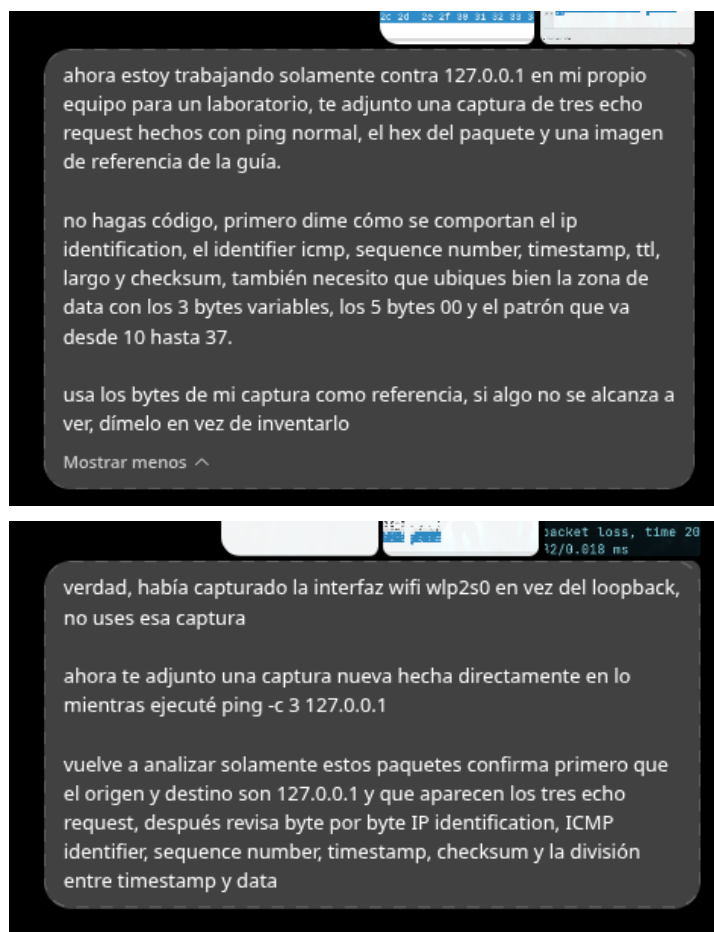


Figura 21: Análisis inicial arriba y corrección de la interfaz abajo.

2. Seleccioné un echo reply. La segunda captura ya era correcta, pero el paquete que abrí tenía ICMP tipo 0. Para poder comparar solicitudes filtré `icmp.type == 8` y elegí echo request antes y después del mensaje. Las figuras 9 y 16 corresponden a esos paquetes corregidos.

3. La primera corrección de DF no cambió el archivo. Al comparar los paquetes encontré que el ping normal usaba DF y el programa usaba `flags=0`. Pedí el cambio mostrado arriba en la figura 22, pero al descargar y repetir la captura el archivo todavía tenía el valor anterior. Envié un segundo mensaje pidiendo el código completo dentro de la respuesta. Después revisé la línea antes de ejecutar otra captura. Ese segundo mensaje aparece abajo.

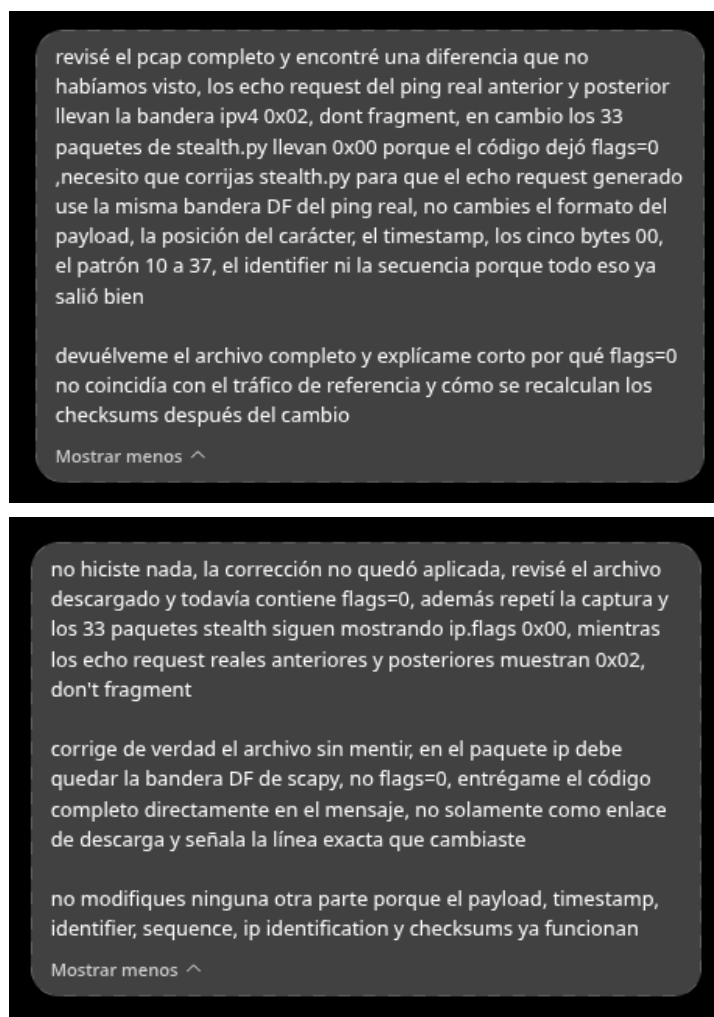


Figura 22: Primer intento de corrección arriba y mensaje que produjo flags="DF" abajo.

4. No había guardado el prompt exacto de stealth. Tenía las capturas del análisis y el código, pero no el mensaje que había usado para pedir el archivo. Volví al historial y recuperé el prompt completo. La captura está en la figura 10 y también dejé una copia de texto en la carpeta de prompts. Desde ese momento guardé cada mensaje antes de probar una corrección.

Conclusiones y comentarios

El programa César entregó `larycxpajorj h bnpdarmjm nw anmnb` para el texto solicitado y el corrimiento 9. Stealth distribuyó sus 33 caracteres en la misma cantidad de echo request. El bloque mantuvo TTL 64, largo IPv4 de 84 bytes, DF, un Identifier fijo y números de secuencia consecutivos. Los intervalos tuvieron un promedio de 1,051 segundos. Wireshark validó los checksums IPv4 e ICMP de todos los paquetes. MitM reconstruyó el criptograma, imprimió las 26 posibilidades y marcó la llave 9.

A simple vista, los echo request generados se parecen a los del ping usado como referencia. Wireshark mostró una diferencia en el aviso [**No response seen**]. Los paquetes normales recibieron echo reply y los 33 paquetes del mensaje no. Con esa diferencia no afirmaré que el tráfico completo sea indistinguible ni que pueda superar un DPI real. La prueba solo comprobó los campos revisados en este laboratorio.

La exigencia de generar los programas con ChatGPT me resultó decepcionante y desmotivante. Ya había trabajado ICMP, Wireshark y análisis de paquetes en Taller de Redes y Servicios como estudiante y ayudante, por lo que no encontré un aprendizaje técnico nuevo. La mayor dificultad fue mantener el material ordenado y revisar cada entrega de ChatGPT. Durante la corrección de DF afirmó haber cambiado el archivo, pero devolvió el mismo código. Alguien sin experiencia previa podría copiar ese resultado sin darse cuenta. En un trabajo de ciberseguridad nunca confiaría a ciegas en código que no entiendo o que no he comprobado.

César me había parecido entretenido al resolverlo a mano. Pedirlo a ChatGPT volvió esa parte rutinaria. MitM también fue directo porque solo debía probar 26 llaves y terminó en segundos. En general quedé con una sensación amarga. Espero que los próximos laboratorios tengan más trabajo propio y que ChatGPT quede como apoyo.